

Politechnika Białostocka
Wydział Informatyki

Finansista PB

System obsługi wniosków o finansowanie
działalności studenckiej i doktoranckiej

Dokumentacja projektowa

Autorzy:

Jakub Matusiewicz
Jakub Borkowski

Przedmioty: Aplikacje webowe w Javie, Systemy baz danych

Białystok, 14 czerwca 2026

Spis treści

1	Wstęp	2
1.1	Cel projektu	2
1.2	Zakres	2
1.3	Zespół i podział pracy	2
2	Wymagania	2
2.1	Wymagania funkcjonalne	2
2.2	Wymagania niefunkcjonalne	3
3	Architektura systemu	3
3.1	Schemat warstw	4
3.2	Stos technologiczny	4
4	Model danych	4
4.1	Diagram encji	4
4.2	Główne tabele	5
4.3	Klucze hybrydowe	5
4.4	Migracje Flyway	5
5	Logika w bazie danych	6
5.1	Maszyna stanów wniosku	6
5.2	Widoki	6
5.3	Indeksy i wydajność	6
6	REST API	6
6.1	Najważniejsze endpointy	7
6.2	Specyfikacja OpenAPI	7
7	Bezpieczeństwo	7
7.1	Uwierzytelnianie	7
7.2	Autoryzacja oparta na rolach	7
7.3	Zabezpieczenia formularzy	8
8	Frontend i dostępność	8
8.1	Dostępność	8
9	Testy wydajnościowe	8
10	Uruchomienie systemu	8
10.1	Wymagania	8
10.2	Krok po kroku	9
10.3	Konta testowe (DataSeeder)	9
11	Podsumowanie	9

1 Wstęp

1.1 Cel projektu

Celem projektu jest dostarczenie systemu informatycznego usprawniającego proces składania, oceny i rozliczania wniosków o finansowanie działalności studenckiej i doktoranckiej w Politechnice Białostockiej. System realizuje papierowy obieg wniosku opisany w *Zarządzeniu nr 13/2025 Rektora Politechniki Białostockiej z dnia 14 lutego 2025 r. w sprawie wydatkowania środków finansowych przeznaczonych na działalność studencką i doktorancką*, ze szczególnym uwzględnieniem *Załącznika nr 1* (wzór formularza wniosku).

1.2 Zakres

System obejmuje:

- rejestrację oraz logowanie użytkowników z rozróżnieniem ról,
- składanie wniosku w wersji elektronicznej w pełni odwzorowującej Załącznik nr 1,
- walidację formularza (pola obowiązkowe, spójność dat i kwot, suma źródeł finansowania),
- obieg statusów wniosku (maszyna stanów po stronie bazy danych),
- podgląd statusu i szczegółów wniosku przez wnioskodawcę,
- REST API jako warstwę komunikacji frontend–backend,
- dokumentację OpenAPI dla wystawionych usług,
- logikę składowaną w bazie danych (pakiety, widoki, wyzwalacze, funkcje),
- zgodność z wymaganiami dostępności (deklaracja dostępności).

1.3 Zespół i podział pracy

- **Jakub Matusiewicz** – warstwa bazy danych (Oracle, PL/SQL, migracje Flyway), frontend (Thymeleaf, Bootstrap), integracja widoków z REST API.
- **Jakub Borkowski** – backend Spring Boot, model encji JPA, use case’y, REST API, bezpieczeństwo (Spring Security + JWT).

2 Wymagania

2.1 Wymagania funkcjonalne

1. Rejestracja i logowanie użytkownika.
2. Tworzenie wniosków w wersji elektronicznej.
3. Wypełnianie wniosku poprzez formularz odwzorowujący Załącznik nr 1.
4. Walidacja formularza.

5. Wysyłanie wniosku do wyższych jednostek (zmiana statusu DRAFT → SUBMITTED).
6. Kontrola wniosku finansowego poprzez sprawdzanie jego statusu.
7. Edycja/poprawa wniosku w razie odrzucenia lub uwag.
8. Procedowanie wniosku przez wyższe jednostki (Dziekanat, CSSDiR, Kwestor) – pola dostępne dla odpowiednich ról oraz odpowiednie ścieżki dla konkretnych wniosków.
9. Dostępność aplikacji dla osób ze specjalnymi potrzebami.
10. Filtrowanie i wyszukiwanie wniosków.
11. Zarządzanie uprawnieniami i rolami użytkowników (administrator).
12. Wyświetlanie pełnej historii zmian statusu wniosku.
13. Źródło finansowania jako zmienna w encji wniosku.

2.2 Wymagania niefunkcjonalne

- **Architektura:** klient–serwer; frontend działa niezależnie od backendu i komunikuje się z nim wyłącznie poprzez REST API.
- **Wydajność:** poprawna praca przy zbiorach 1 000–1 000 000 wniosków (z paginacją i odpowiednimi indeksami).
- **Bezpieczeństwo:** uwierzytelnianie tokenem JWT przechowywanym w bezpiecznym ciasteczku `HttpOnly`, autoryzacja oparta na rolach.
- **Dostępność:** zgodność z WCAG 2.1 – skip link, semantyczne nagłówki, kontrast, etykiety formularzy, opublikowana deklaracja dostępności.

3 Architektura systemu

System jest aplikacją monolityczną logicznie podzieloną na trzy warstwy:

- **Warstwa prezentacji (frontend)** – kontrolery Spring MVC zwracające widoki Thymeleaf renderowane po stronie serwera. Frontend *nie woła* warstwy biznesowej bezpośrednio – komunikuje się z backendem wyłącznie przez REST API z użyciem `RestClient`, przekazując token JWT.
- **Warstwa REST API (backend)** – kontrolery `@RestController` wystawiające usługi pod ścieżką `/api/v1/...` (JSON). Stanowi kontrakt komunikacyjny niezależny od użytego klienta.
- **Warstwa danych** – baza Oracle z migracjami Flyway, logiką składowaną (PL/SQL) oraz dostępem przez JPA/Hibernate.

3.1 Schemat warstw

Miejsce na schemat warstw
(Przeglądarka $\rightarrow$ Kontrolery MVC $\rightarrow$ REST API $\rightarrow$ Use case'y $\rightarrow$ JPA $\rightarrow$ Oracle)

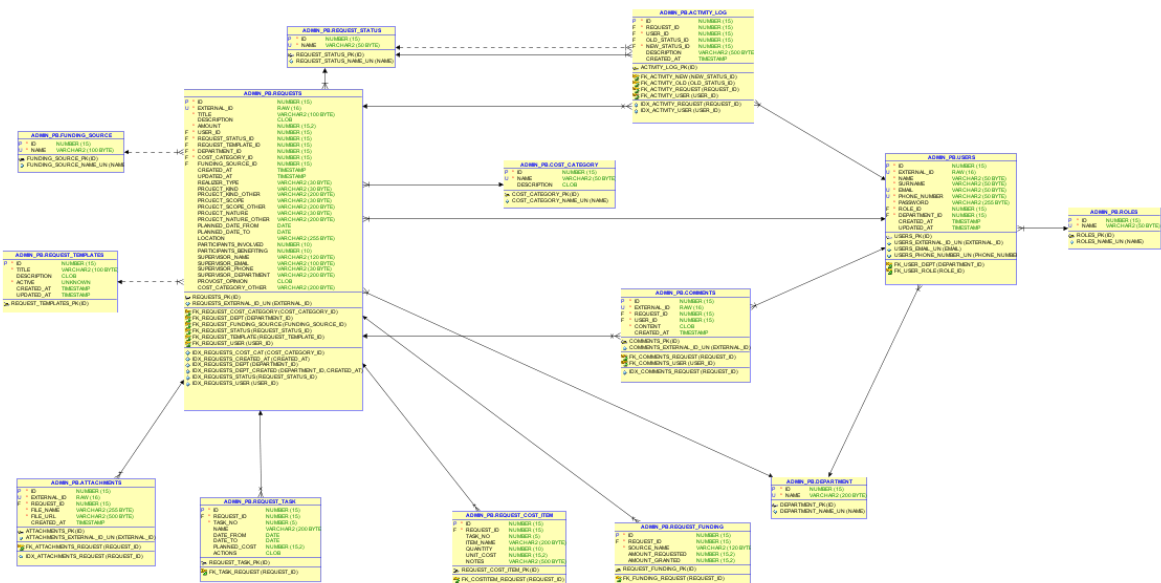
Rysunek 1: Schemat warstw aplikacji.

3.2 Stos technologiczny

- Java 25, Spring Boot 4.0
- Spring MVC, Thymeleaf 3, Bootstrap 5
- Spring Security, JWT 0.12
- Hibernate / Spring Data JPA
- Flyway (migracje)
- Oracle Database 23 Free (kontener Docker)
- Lombok, springdoc-openapi (Swagger UI)
- Build: Maven

4 Model danych

4.1 Diagram encji



Rysunek 2: Diagram encji

4.2 Główne tabele

- `users` – konta użytkowników; powiązane z `roles` i `department`.
- `requests` – główna encja wniosku (sekcje I–III Załącznika nr 1).
- `request_task` – pozycje harmonogramu (sekcja IV).
- `request_cost_item` – pozycje kosztorysu (sekcja IV).
- `request_funding` – źródła finansowania z kwotą wnioskowaną i przyznaną (sekcja VI).
- `request_status` – słownik statusów wniosku.
- `activity_log` – historia zmian statusu.
- `comments`, `attachments` – komentarze i załączniki do wniosków.
- Słowniki: `roles`, `department`, `cost_category`, `funding_source`, `request_templates`.

4.3 Klucze hybrydowe

Każda eksponowana encja używa dwóch identyfikatorów: wewnętrznego `id` (`NUMBER IDENTITY`) używanego w kluczach obcych oraz zewnętrznego `external_id` (`RAW(16)` – `UUID v7`), który jest jedynym identyfikatorem wystawianym w REST API i adresach URL. Zapobiega to wyciekowi informacji o liczności rekordów i utrudnia enumerację zasobów.

4.4 Migracje Flyway

- V1 – inicjalny schemat (11 tabel: `users`, `roles`, `department`, `requests`, ...).
- V2 – wypełnienie słowników (role, działy, statusy, kategorie).
- V3 – widoki bazodanowe (m.in. `v_department_requests_summary`).
- V4 – wyzwalacze (audyt, walidacje).
- V5 – pakiet PL/SQL `pkg_finansista` (procedury domenowe).
- V6 – indeksy wydajnościowe.
- V7 – maszyna stanów wniosku (funkcja + trigger blokujący niedozwolone przejścia).
- V8 – rozszerzenie schematu pod Załącznik nr 1 (sekcje I–III + tabele `request_task`, `request_cost_item`, `request_funding`).
- V9 – dodanie jednostki *Samorząd Doktorantów PB*.
- V10 – kategoria *Inne* + pole opisowe `cost_category_other`.

5 Logika w bazie danych

5.1 Maszyna stanów wniosku

Dozwolone przejścia statusów są egzekwowane na poziomie bazy (V7):

Status	Dozwolone przejścia
DRAFT	SUBMITTED
SUBMITTED	FORMAL_EVALUATION, REJECTED, CORRECTION_REQUIRED
FORMAL_EVALUATION	UNDER_REVIEW, REJECTED, CORRECTION_REQUIRED
UNDER_REVIEW	ACCEPTED, REJECTED, CORRECTION_REQUIRED
CORRECTION_REQUIRED	DRAFT, SUBMITTED

Próba zmiany statusu na niedozwolony skutkuje błędem `ORA-20002`.

5.2 Widoki

- `v_department_requests_summary` – podsumowanie liczby i kosztu wniosków per wydział (na potrzeby raportowania administratora).

5.3 Indeksy i wydajność

Po stronie bazy założono indeksy na kolumnach najczęściej używanych w filtrach (`request_status_id`, `department_id`, `user_id`, `created_at`). Wyniki testów wydajnościowych dla zbiorów 1 k, 10 k, 100 k oraz 1 M wierszy znajdują się w pliku `docs/testy-wydajnosciowe.md`.

6 REST API

Frontend Thymeleaf komunikuje się z backendem wyłącznie poprzez REST API, co wymusza poprawną architekturę klient-serwer. Token JWT z ciasteczka przeglądarki jest przekazywany jako nagłówek `Authorization: Bearer`.

6.1 Najważniejsze endpointy

Metoda	Ścieżka	Opis
POST	/api/v1/auth/register	rejestracja konta
POST	/api/v1/auth/login	logowanie (ustawia ciasteczko JWT)
POST	/api/v1/auth/logout	wylogowanie (kasuje ciasteczko)
GET	/api/v1/requests	lista wniosków (filtr po statusie/wydziale)
POST	/api/v1/requests	utworzenie wniosku (pełny Załącznik nr 1)
GET	/api/v1/requests/{id}	szczegóły wniosku
PUT	/api/v1/requests/{id}	edycja wniosku
PATCH	/api/v1/requests/{id}/status	zmiana statusu
GET	/api/v1/requests/{id}/history	historia zmian statusu
POST	/api/v1/requests/{id}/comments	dodanie komentarza
GET	/api/v1/requests/{id}/comments	komentarze
DELETE	/api/v1/requests/{id}	usunięcie wniosku (admin)

6.2 Specyfikacja OpenAPI

Pełna specyfikacja jest udostępniona przez Swagger UI pod adresem <http://localhost:8080/swagger-ui/index.html> oraz w formie JSON pod <http://localhost:8080/v3/api-docs>.

7 Bezpieczeństwo

7.1 Uwierzytelnianie

Po poprawnym zalogowaniu backend wystawia token JWT podpisany kluczem HMAC SHA-256 i zwraca go w bezpiecznym ciasteczku z atrybutami `HttpOnly`, `SameSite=Strict`. Token zawiera identyfikator użytkownika (e-mail) oraz jego rolę.

7.2 Autoryzacja oparta na rolach

- `ROLE_STUDENT` – składa wnioski, widzi własne;
- `ROLE_WRSS` – ocena merytoryczna w ramach Samorządu;
- `ROLE_LEGAL_COMMISSION` – ocena merytoryczna w komisji;
- `ROLE_DEAN_OFFICE` – ocena formalna (CSSDiR/Dziekanat);
- `ROLE_FINANCE_OFFICE` – Kwestura (rozliczenie);
- `ROLE_ADMIN` – decyzja Rektora, zarządzanie systemem.

7.3 Zabezpieczenia formularzy

Rola użytkownika *nigdy* nie jest pobierana z formularza rejestracji – każde nowe konto otrzymuje rolę bazową `ROLE_STUDENT` wymuszoną po stronie serwera. Przypisanie wyższej roli jest wyłącznie aktem administracyjnym. Podobnie wydział wnioskodawcy nie jest wybierany przy składaniu wniosku – pobierany jest z konta zalogowanego użytkownika.

8 Frontend i dostępność

Warstwa prezentacji oparta jest o szablony Thymeleaf z motywem Politechniki Białostockiej (kolorystyka zielona, flat design, Bootstrap 5). Każdy widok dziedziczy po wspólnym układzie `layout/base.html` zawierającym nawigację i stopkę.

8.1 Dostępność

- Skip link na początku każdego widoku.
- Semantyczne nagłówki i etykiety formularzy.
- Kontrast kolorów zgodny z WCAG 2.1 AA.
- Opublikowana *Deklaracja dostępności* pod adresem `/accessibility`.
- Strony błędów 400/403/404/500 w spójnym stylu HTML (nie JSON).

9 Testy wydajnościowe

Wykonano testy wydajnościowe na zbiorach 1 000, 10 000, 100 000 oraz 1 000 000 wierszy w tabeli `requests`. Sprawdzano:

- czas pobrania pojedynczego wniosku po `external_id`,
- czas listowania wniosków z paginacją,
- czas zmiany statusu (kluczowa operacja transakcyjna),
- plany wykonania zapytań przed i po dodaniu indeksów (V6).

Szczegółowe wyniki oraz porównanie planów wykonania zostały umieszczone w pliku `docs/testy-wydajnosciowe.md`.

10 Uruchomienie systemu

10.1 Wymagania

- Java 25, Maven 3.9+
- Docker (do uruchomienia Oracle Database 23 Free)

10.2 Krok po kroku

```
1 # 1. Uruchomienie bazy Oracle w Dockerze
2 docker run -d --name oracle-pb -p 1521:1521 \
3   -e ORACLE_PASSWORD=pawlewjavle \
4   gvenzl/oracle-free:23-slim
5
6 # 2. Utworzenie schematu admin_pb (z hasłem zgodnym z application.yaml)
7 #   - skrypt w docs/install/create_schema.sql lub ręcznie w SQL
8   Developer
9
10 # 3. Uruchomienie aplikacji
11 ./mvnw spring-boot:run
12
13 # 4. Aplikacja dostępna pod:
14 #   http://localhost:8080
```

Kod 1: Uruchomienie aplikacji

10.3 Konta testowe (DataSeeder)

E-mail	Hasło	Rola / jednostka
j.wnioskodawca@student.pb.edu.pl	student123	STUDENT / Wydział Informatyki
k.samorzad@pb.edu.pl	wrss123	WRSS / Samorząd Studentów
a.zgodna@pb.edu.pl	komisja123	LEGAL_COMMISSION / Wydział Mechaniczny
e.dziekan@pb.edu.pl	dziekanat123	DEAN_OFFICE / Dziekanat WI
j.matusiewicz@student.pb.edu.pl	admin123	ADMIN / Wydział Mechaniczny
j.borkowski@student.pb.edu.pl	admin123	ADMIN / Wydział Informatyki

11 Podsumowanie